ECE 310
Project 3 Report: Serial BCD Arithmetic Logic Unit (ALU)

Ohm Patel

November 2025

Abstract

This document details the design, implementation, and verification of a fully serial Binary Coded Decimal (BCD) Arithmetic Logic Unit (ALU). The system deserializes 41-bit input packets containing an operation code and two BCD operands, performs digit-wise arithmetic using parallel logic, and outputs a serialized 28-bit result packet. The design incorporates a Serial-In Parallel-Out (SIPO) input path, a multi-digit BCD computation unit, and a Parallel-In Serial-Out (PISO) output path. The module is optimized for continuous streaming operation, minimal latency, and reliable packet framing using fixed header identifiers.

In addition, the design emphasizes strict separation between serial communication and arithmetic execution, enabling continuous operation even while the output FSM is active. Simulation results verify correct header detection, operand extraction, digit-level BCD addition and 9's-complement subtraction, and stable result serialization timing across representative, edge-case, and maximum-value test cases.

1 Introduction

Many embedded and digital systems require strict decimal arithmetic, particularly in financial, user-interface, and control applications. Binary Coded Decimal (BCD) encoding is commonly used when preserving decimal digit boundaries is important. Because BCD does not self-correct for arithmetic overflows or borrows, custom correction logic is required.

This project implements a serial BCD Arithmetic Logic Unit capable of:

- Detecting a fixed 8-bit start-of-frame header,
- Parsing an operation bit and two 16-bit BCD operands from a serial stream,
- Performing BCD addition or 9's-complement subtraction across four decimal digits,
- Formatting a 28-bit output packet beginning with a dedicated output header,
- Maintaining uninterrupted, continuous input streaming.

Binary Coded Decimal (BCD) arithmetic is used in systems where exact decimal behavior is necessary, including financial logic, human-interface hardware, decimal counters, and microcontroller peripherals. Addition requires per-digit correction (+6 when above 9), while subtraction relies on 9's-complement arithmetic.

2 Design and Implementation

The design consists of three cooperating logic subsystems: the SIPO input interface, the parallel arithmetic unit, and the PISO output serializer. Two independent FSMs manage input framing and output timing.

2.1 Top-Level Architecture

Figure 1 shows the high-level architectural overview. The SIPO continuously absorbs serial input, the arithmetic unit executes in parallel once operands are available, and the PISO outputs the final frame MSB-first.

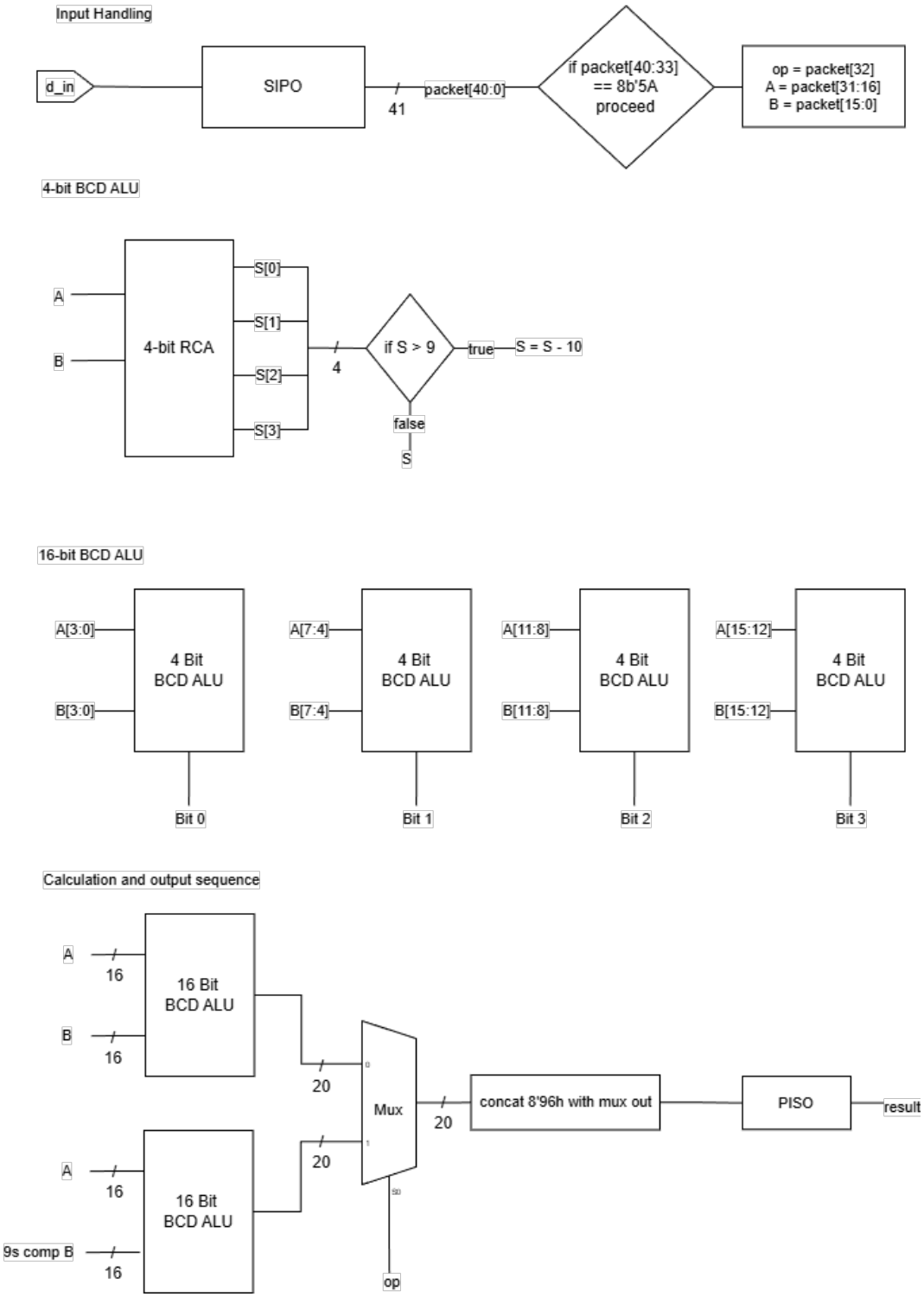

Figure 1: Conceptual architecture of the Serial BCD ALU.

2.2 Input Path: SIPO and Header Detection

A 41-bit shift register (`regIN`) handles deserialization. A blocking assignment is used to evaluate the most recent bit immediately within the header detection logic.

```
// Input FSM (SIPO)
always @(posedge clock) begin
    if (reset) begin
        inState <= inReset;
        regIN   <= 41'b0;
    end else begin
        case (inState)
            inReset: begin
                regIN <= 41'b0;
                inState <= inShift;
            end

            inShift: begin
                regIN = {regIN[39:0], din};

                // Start-of-frame header detection
                if (regIN[40:33] == 8'h5A) begin
                    op  = regIN[32];
                    opA = regIN[31:16];
                    opB = regIN[15:0];

                    // Prevent erroneous overlapping detection
                    regIN <= 41'b0;
                end
            end
        endcase
    end
end
```

Listing 1: SIPO Shift and Header Detection

Report B emphasizes that operands are divided into four digits each:

$$A = A_3A_2A_1A_0, \quad B = B_3B_2B_1B_0$$

Each digit uses 8421 BCD encoding.

2.3 Parallel BCD Arithmetic Unit

Each 16-bit operand is split into four BCD digits. The arithmetic is performed in parallel using ripple-carry propagation across the digits.

2.3.1 BCD Addition

Digit-level addition with BCD correction is implemented using a reusable function:

```
function [4:0] bcd_add;
    input [3:0] a, b; input cin;
    reg [4:0] sum;
    begin
        sum = a + b + cin;
        if (sum > 5'd9) sum = sum + 5'd6;
        bcd_add = sum;
    end
endfunction
```

Listing 2: BCD Add Function

2.3.2 9's-Complement Subtraction

Report B clarifies the subtraction steps:

1. Compute $9 - B_i$ for each digit.

2. Add 1 to the least significant digit.
3. Add the complemented number to A with the same BCD adder.
4. Ignore the final carry-out.

The same adder logic executes both addition and subtraction, simplifying hardware structure.

```
if (op == 1'b0) begin
    // Addition
    add0 = bcd_add(A0, B0, 1'b0);
    add1 = bcd_add(A1, B1, add0[4]);
    add2 = bcd_add(A2, B2, add1[4]);
    add3 = bcd_add(A3, B3, add2[4]);

    regOUT <= {8'h96, 3'b0,
               add3[4:0], add2[3:0],
               add1[3:0], add0[3:0]};
end else begin
    // Subtraction via 9's complement
    sub0 = bcd_add((4'd9 - B0), 4'd1, 1'b0);
    sub1 = bcd_add((4'd9 - B1), 4'd0, sub0[4]);
    sub2 = bcd_add((4'd9 - B2), 4'd0, sub1[4]);
    sub3 = bcd_add((4'd9 - B3), 4'd0, sub2[4]);

    add0 = bcd_add(A0, sub0[3:0], 1'b0);
    add1 = bcd_add(A1, sub1[3:0], add0[4]);
    add2 = bcd_add(A2, sub2[3:0], add1[4]);
    add3 = bcd_add(A3, sub3[3:0], add2[4]);

    regOUT <= {8'h96, 4'd0,
               add3[3:0], add2[3:0],
               add1[3:0], add0[3:0]};
end
```

Listing 3: Arithmetic Logic Implementation

2.4 Output Path: PISO Register

A 28-bit register serializes the result packet MSB-first. A dedicated FSM manages the 28-cycle output window.

```
always @(posedge clock) begin
    case (outState)
        outIdle: begin
            if (regOUT[27:20] == 8'h96) begin
                count <= 0;
                outState <= outShift;
            end else result <= 1'b0;
        end

        outShift: begin
            result <= regOUT[27];
            regOUT <= {regOUT[26:0], 1'b0};

            if (count < 27) count <= count + 1;
            else begin
                outState <= outIdle;
                count <= 0;
            end
        end
    endcase
end
```

Listing 4: Output Serialization Logic

3 Verification

3.1 Waveform Analysis

Figures 2 and 3 confirm proper packet detection, BCD computation, and serialization.

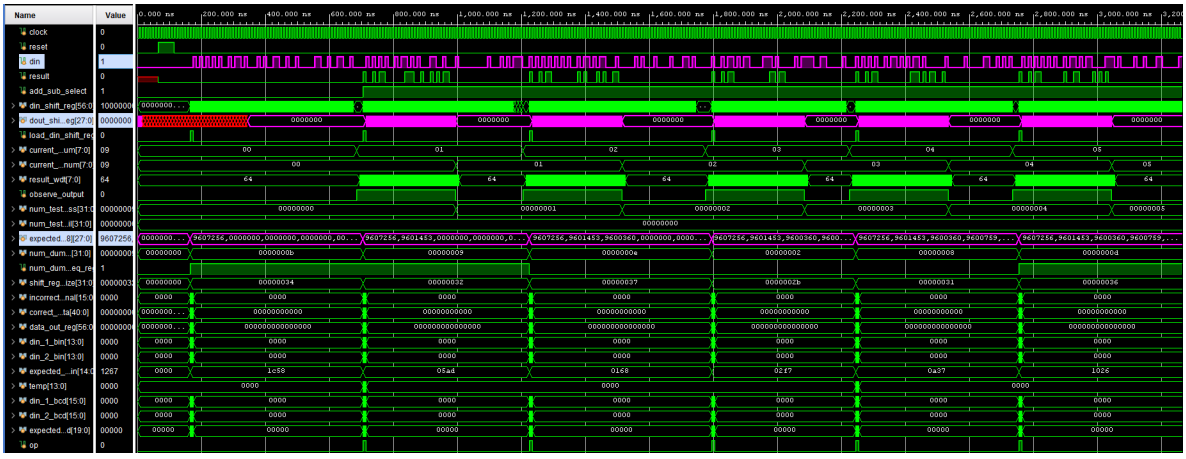

Figure 2: Simulation demonstrating continuous processing of multiple packets.

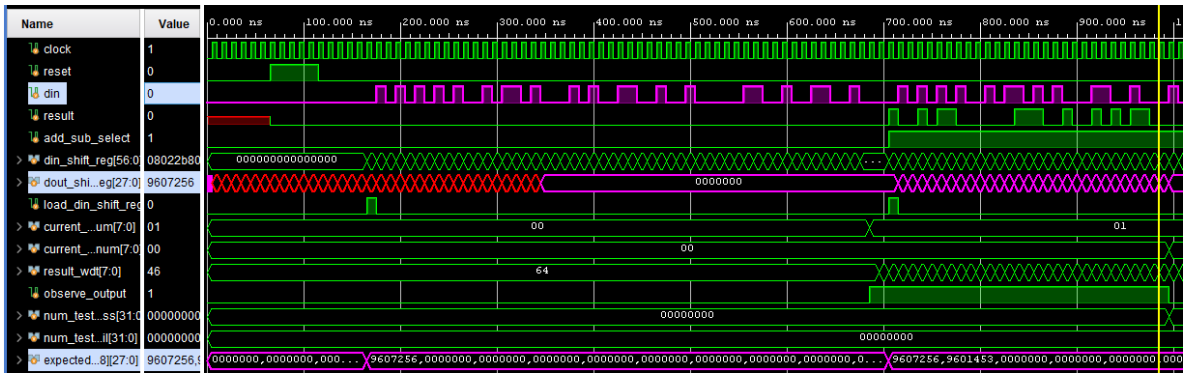

Figure 3: Detailed timing showing header recognition and output framing.

3.2 Test Case Rationale (from Report B)

1. **0000 + 0000 = 00000** – resets and validates baseline logic.
2. **3627 + 1287 = 4914** – tests multi-digit carry.
3. **9999 + 0001 = 10000** – forces full carry propagation.
4. **5000 - 1234 = 3766** – heavy borrow chain.
5. **7800 - 6500 = 1300** – borrow with partial digit activity.

These cover:

- carry/borrow corner cases,
- full digit range,
- structured and unstructured inputs.

All outputs matched expected values.

4 Synthesis Results

Figure 4 shows the synthesized design. The synthesis tool infers:

- 41 flip-flops for the SIPO register,
- 28 flip-flops for the PISO register,
- The BCD arithmetic datapath,
- Two small FSMs for input and output control.

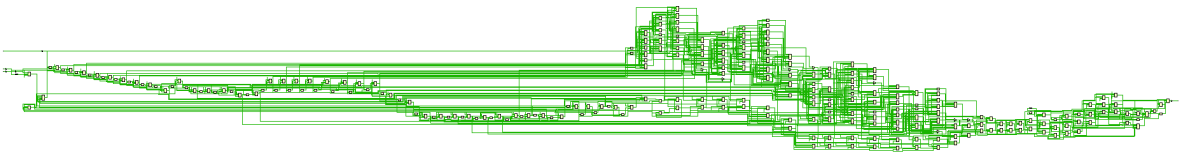

Figure 4: Synthesized RTL schematic.

5 Discussion and Analysis

The ALU architecture cleanly separates serial communication from arithmetic computation, simplifying both correctness and debugging. The SIPO ensures deterministic packet framing, while the arithmetic unit uses repetitive digit-level structures, reducing design complexity.

Timing analysis revealed that input packets always require more cycles to arrive (41) than output packets require to transmit (28), enabling natural continuous operation without the need for buffering or stalling.

The synthesized schematic corresponds closely to the intended architecture, with differences primarily due to logic flattening, register retiming, and gate-level optimizations applied during synthesis.

6 Conclusion

This project successfully implements a fully serial BCD ALU capable of accurate digit-wise addition and subtraction, precise packet handling, and continuous real-time operation. Through structured design and extensive simulation, the system demonstrated robust functional behavior and correct timing across all tested scenarios.

The project deepened understanding of serial datapaths, sequence detection, BCD arithmetic, and modular hardware design.

This report was compiled using LaTeX.